# Mimicry: A SynthID-Inspired Invisible Image Watermarking System

Mehdi Ait Abdelouahab

April 19, 2026

**Abstract**

Mimicry is a compact invisible image watermarking system implemented in PyTorch and designed as an honest, research-style attempt to mimic the broad training philosophy of Google SynthID. The project couples an encoder, which hides a binary key inside an image, with a decoder, which attempts to recover that key after image degradations. The final system was trained with a staged pipeline that moved from clean supervision to robustness-focused optimization under differentiable attacks. On the final robust 16-bit checkpoint, Mimicry achieved an average attacked bit accuracy of 0.6465 across clean, JPEG, blur, crop, noise, and brightness conditions, with 23.52 dB PSNR and 0.8642 SSIM on the held-out test split. The results show meaningful above-random watermark recovery, but also expose the central robustness-quality tradeoff and the gap between partial bit recovery and exact full-key recovery.

## 1 Introduction

Invisible watermarking aims to encode information into an image in a way that remains visually unobtrusive while still being recoverable after common post-processing operations. The practical challenge is that robustness and imperceptibility pull in opposite directions: a stronger watermark is easier to recover, but also easier to see or more likely to degrade image quality.

Mimicry was built as a small, inspectable system inspired by the public idea of SynthID-style training rather than as a claim of faithful reproduction. The goal was to implement the core ingredients from scratch:

- an encoder that writes a binary watermark into an image,
- a decoder that predicts the hidden key,
- a differentiable augmentation pipeline that simulates distortions during training,
- and an evaluation protocol that separates image quality from robustness.

## 2 Method

### 2.1 Architecture

The system consists of two neural networks.

**Encoder.** The encoder is a UNet-lite architecture with three downsampling blocks and three upsampling blocks. The binary watermark key is spatially broadcast and concatenated at the bottleneck. The model predicts a residual image bounded by a scaled hyperbolic tangent, and the final watermarked image is produced by adding the residual to the original image followed by clamping to $[0, 1]$.

**Decoder.** The decoder is a lightweight ResNet-style network with a convolutional stem, four residual blocks, two downsampling stages, global average pooling, and a final linear projection to watermark logits.

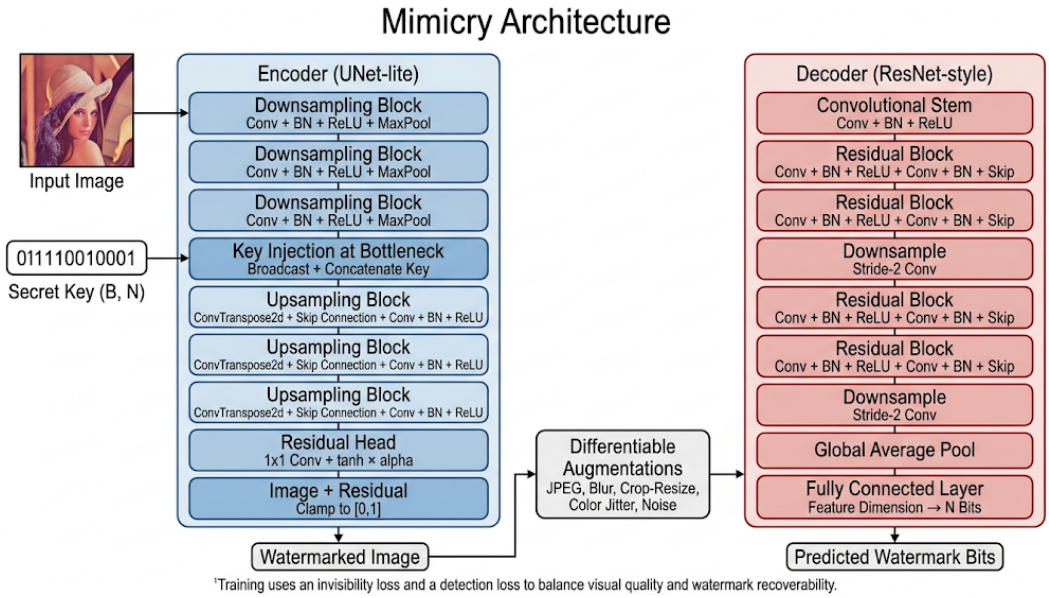


Figure 1: Encoder-Decoder Architecture

## 2.2 Training Objective

The training loss combines two terms:

$$\mathcal{L}_{total} = \lambda_{inv}\mathcal{L}_{inv} + \lambda_{det}\mathcal{L}_{det} \tag{1}$$

where:

- $\mathcal{L}_{inv}$ is the invisibility term, implemented as a weighted combination of $(1 - \text{SSIM})$ and L2 reconstruction,
- $\mathcal{L}_{det}$ is the binary cross-entropy loss between predicted watermark logits and target bits.

## 2.3 Attack Model

During robust training, the watermarked image may be passed through differentiable approximations of:

- JPEG compression,
- Gaussian blur,
- crop-resize,
- color jitter,
- additive Gaussian noise.

# 3 Experimental Setup

## 3.1 Data

The experiments used a lightweight COCO-based split:

- 4000 training images,
- 500 validation images,
- 500 test images.

Images were cropped to a fixed resolution of $128 \times 128$.

## 3.2 Training Stages

The final reported checkpoint came from a staged training process:

1. **Clean bootstrap.** A no-attack training phase established the basic embedding and decoding behavior.
2. **Robust stage 1.** Attacks were enabled while keeping the optimization relatively balanced.
3. **Robustness-focused continuation.** The watermark strength and detection weight were increased, while invisibility pressure was reduced.

This last regime change is visible in the training plots as a sharp jump in loss followed by renewed optimization. That jump is not a bug. It reflects a real change in the objective scale after the model was pushed into a stronger robustness-first operating point.

## 3.3 Payload

The final reported model used a 16-bit payload. That means each image carries a binary vector of length 16, and the decoder predicts 16 probabilities which are thresholded to bits during evaluation.

# 4 Results

## 4.1 Quantitative Evaluation

The final robust checkpoint achieved the following test-set results:

| Condition | Bit Accuracy | Exact Match |
|---|---|---|
| Clean | 0.6528 | 0.0020 |
| JPEG | 0.6536 | 0.0000 |
| Blur | 0.6472 | 0.0000 |
| Crop | 0.6300 | 0.0020 |
| Noise | 0.6503 | 0.0020 |
| Brightness | 0.6450 | 0.0000 |
| Average | 0.6465 | 0.0010 |

Table 1: Final robustness evaluation for the selected 16-bit checkpoint.

Image-quality metrics for the same checkpoint were:

- PSNR: 23.52 dB
- SSIM: 0.8642

## 4.2 Training Curves

Figure 2 summarizes the main training metrics. Figure 3 isolates watermark bit accuracy.

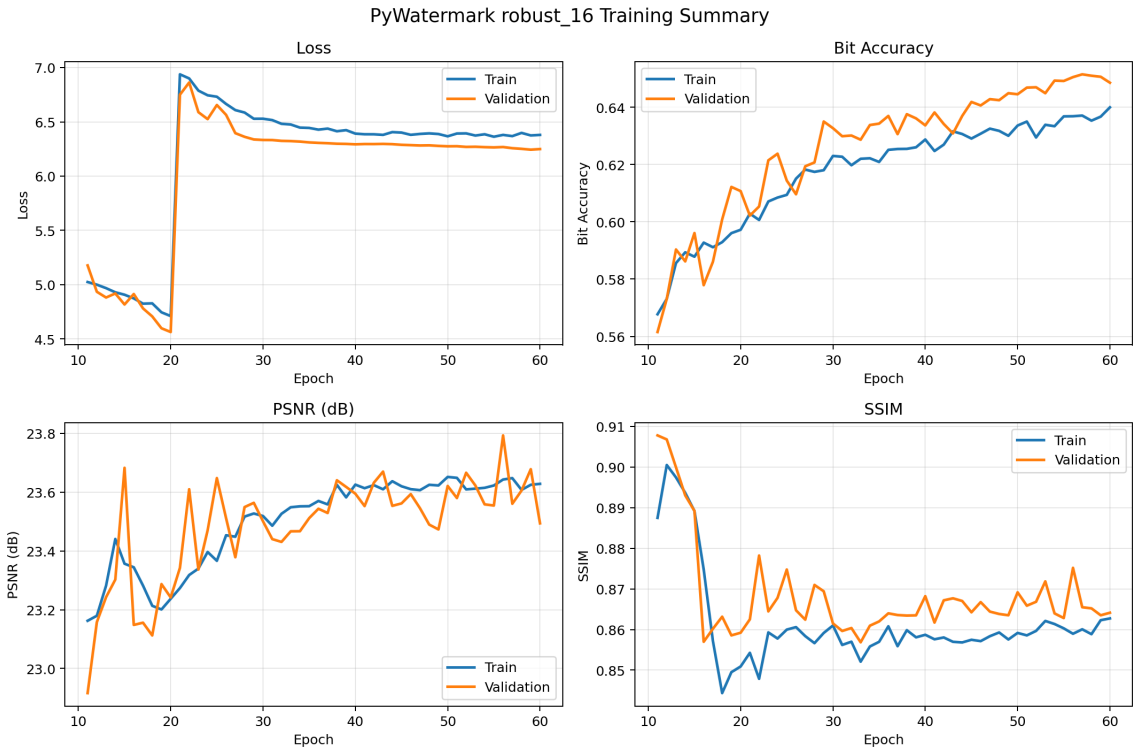


Figure 2: Training summary for the final robust 16-bit run. The jump in loss marks the transition into a stronger robustness-focused regime.

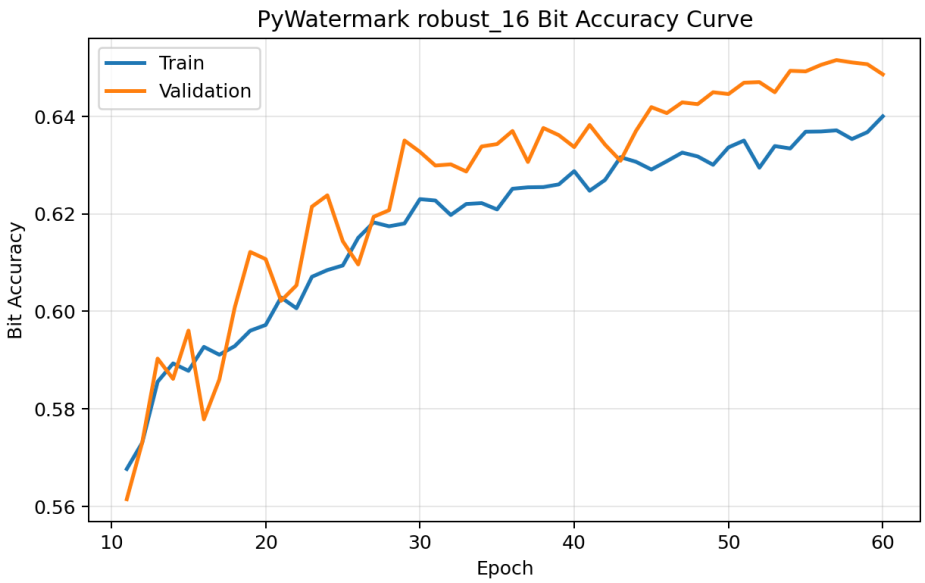


Figure 3: Train and validation bit accuracy across epochs. Accuracy improves steadily even after the regime change.

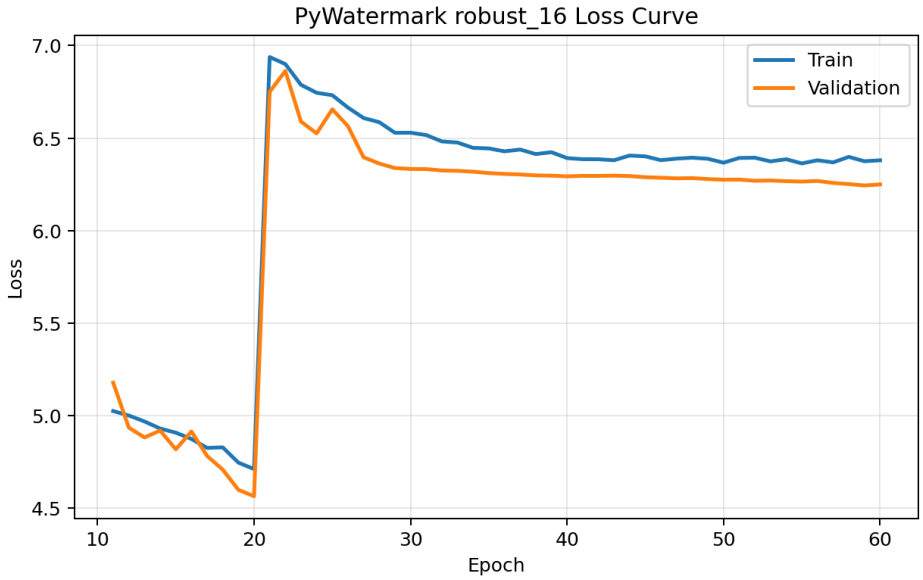


Figure 4: Loss curve across epochs. The large increase after the earlier epochs reflects a deliberate change in loss weighting rather than numeric instability.

### 4.3 figure 4 explanation

The loss discontinuity around epoch 21 corresponds to a deliberate transition into a stronger robustness-oriented training regime. In the first phase, the system was trained with encoder strength $\alpha = 0.10$, invisibility weight $\lambda_{inv} = 0.25$, and detection weight $\lambda_{det} = 8.0$. In the second phase, the encoder strength was increased to $\alpha = 0.15$, the invisibility weight was reduced to $\lambda_{inv} = 0.10$, and the detection weight was increased to $\lambda_{det} = 12.0$. As a result, the scale of the optimization objective changed, so the loss values before and after epoch 21 should not be interpreted as directly comparable magnitudes.

## 5 Discussion

### 5.1 What Worked

The final model is clearly above random guessing. Since the watermark payload is binary, a random decoder would remain near 0.50 bit accuracy. Mimicry reaches roughly 0.65 across several distortions, which means the system learned a meaningful watermark signal rather than collapsing entirely.

The model also behaves consistently across the evaluated attack types. JPEG, noise, and brightness are slightly stronger than blur and crop, but the spread is not extreme. This suggests that the learned representation is not narrowly specialized to one single attack family.

### 5.2 What Did Not Work

The system does not reach strong exact-key recovery. Exact match remains near zero, because recovering all 16 bits correctly for a single image is still much harder than recovering a majority of them. In practice, Mimicry demonstrates partial watermark recovery much more reliably than perfect full-key reconstruction.

The model also pays a substantial image-quality price for its robustness gains. The final PSNR and SSIM indicate a noticeable drop in perceptual fidelity compared with earlier, milder checkpoints.

### 5.3 Why the Final Test Accuracy Is Lower Than Peak Validation Impressions

Several factors likely contribute to the gap between encouraging training-time validation behavior and the final held-out evaluation:

- **Checkpoint selection mismatch.** The training loop selected the best checkpoint using clean validation bit accuracy rather than attacked validation accuracy.
- **Payload difficulty.** A 16-bit payload is still demanding for a compact model at $128 \times 128$.
- **Model capacity.** The encoder and decoder remain relatively small.
- **Dataset scale.** A 4000-image training split is practical for Colab, but small for a robustness-heavy watermarking problem.
- **Attack diversity.** The augmentation mix is useful, but may still be suboptimal for the exact robustness profile tested at evaluation.

## 6 Limitations

This project has several limitations:

- It is inspired by SynthID, not a faithful reproduction of Google's internal method.
- It uses fixed-size square crops and does not preserve arbitrary original image resolution in inference.
- The final model demonstrates moderate rather than strong robustness.
- Exact full-key recovery is still rare.
- Crop robustness remains the weakest tested condition.

## 7 Future Work

The most promising follow-up directions are:

- selecting checkpoints using attacked validation mean instead of clean validation only,
- increasing decoder capacity further,
- exploring smaller payloads such as 8 bits for higher clean reliability,
- improving crop-specific robustness,
- training on larger and more diverse image collections,
- and using a more deliberate training curriculum with progressively harder attacks.

## 8 Conclusion

Mimicry demonstrates that a small, fully inspectable PyTorch system can learn a non-trivial invisible watermarking behavior under common distortions. The final model does not match the robustness one would expect from a production-grade watermarking system, but it succeeds as an honest SynthID-inspired research implementation. The experiments clearly expose the core tradeoff: increasing recoverability improves robustness, but pushes image quality downward. In that sense, Mimicry achieves its main goal: it makes the watermarking problem concrete, measurable, and extensible for future work.